

NOVA

IMS

Information
Management
School

MDSAA

Master's Degree Program in
Data Science and Advanced Analytics

Big Data Analytics

Real-Time AI-Powered Fraud Detection on Spark: A Big Data Streaming Solution

Rodrigo Luís, Number: 20240742

João Luís Marques, 20240656

Gustavo Gomes, 20240657

Diogo Ruivo, Number: 20240584

Rafael Lopes, Number: 20240588

Group 17

NOVA Information Management School
Instituto Superior de Estatística e Gestão de Informação

Universidade Nova de Lisboa

2025

INDEX

1. INTRODUCTION.....	2
1.1. Why Fraud Detection Is a Big Data Problem.....	2
1.2. Framing the Problem & Dataset.....	2
2. DATA COLLECTION AND PREPROCESSING.....	2
2.1 Initial Exploration and Data Quality.....	3
2.2 Data Overview and Numeric Feature Analysis.....	3
2.3 Feature Engineering.....	3
2.3.1 Time Derived columns.....	4
2.3.2 Rolling Behavioural metrics.....	4
2.4 Pipeline.....	4
2.5 Feature Selection.....	4
3. METHODOLOGY.....	4
4. RESULTS AND EVALUATION.....	5
4.1 Offline Results.....	5
4.2 Distributed Streaming implementation.....	5
5. CONCLUSION.....	6
Appendix.....	7

1. INTRODUCTION

1.1. Why Fraud Detection Is a Big Data Problem

In assessing whether our fraud-detection use case qualifies as a true Big Data challenge, we framed it against the “three Vs”:

- **Volume.** Financial institutions routinely process millions of transactions per day. Our 5 million-row sample mirrors that scale, and our Spark code is designed to partition, cache, and aggregate data in parallel in order to optimize memory usage.
- **Velocity.** Transactions arrive in real time. By simulating a Kafka → Spark Structured Streaming pipeline, we ensure each new record is ingested, processed and fed into our classification models, with the results being streamed again to kafka for ease of access, for real time analysis
- **Variety.** Beyond the core schema (transaction ID, timestamp, sender/receiver, amount, type), we enriched every event with behavioral and metadata signals, such as geo-anomaly scores derived from IP and device hashes, time-since-last-transaction deltas, spending-deviation metrics, and more. This unified pipeline seamlessly handles numeric, categorical, geospatial, and temporal data.

1.2. Framing the Problem & Dataset

Data Source: A synthetically generated bank transaction log—≈5 million rows, 15+ columns.

Key Features:

- a. **Static:** transaction_id, timestamp, amount, transaction_type
- b. **Behavioral:** time_since_last_txn, velocity_score, deviation_from_mean_spend
- c. **Metadata:** geo_location, device_hash, IP_address, payment_channel

Sampling & Scaling: We down-sample or window our stream for development, but we discuss how partitioning by account and time windows would enable horizontal scaling to billions of records per day.

2. DATA COLLECTION AND PREPROCESSING

To solve this fraud detection challenge, the first step was the sourcing of the “**financial_fraud_detection_dataset**” from Kaggle, which contains **5 million transaction** records from the 1st of January of 2023 to the 1st of January of 2024. We uploaded this data in the .csv format to Databricks, where our analyses were developed and executed, using **PySpark**.

The target variable, naturally, is *is_fraud*, where:

- 1 indicates a fraudulent transaction;
- 0 indicates a legitimate transaction;

Only **3.59%** of the total transactions were fraudulent, which made this a **highly imbalanced** binary classification problem, very common but also a critical challenge in real world fraud detection. [1]

2.1 Initial Exploration and Data Quality

The Exploratory Data Analysis revealed there were **no duplicate** rows but that some column types required some small adjustments to ensure consistency. For example, the column *velocity_score* was stored as an integer, but to maintain consistency across all numeric features, we converted it to double type. [2][3]

We also decided to apply some sanity filters to remove transactions that could have a negative *amount* or invalid *event_time*, but upon further inspection, we realised these columns had always valid values, so at this stage, no rows had been removed yet but the filters were maintained in the preprocessing pipeline as a precaution. [4]

The next issue we noticed was **missing** and **negative** values in *time_since_last_transaction*, which were respectively solved by **imputing the median** and replacing them with the **absolute** values. [5][6]

2.2 Data Overview and Numeric Feature Analysis

To better understand the dataset structure and behaviour, we computed a set of descriptive statistics and quality checks to verify the dataset [7]:

- Still the same 5 000 000 rows;
- Approximately **875 000 unique sender accounts** and **870 000 unique receivers**;
- A **fraud prevalence of 3.59%**;

Then, we evaluated the distribution of the key numerical features in order to detect skewness and make informed decisions regarding transformation. Some findings include [8]:

- *amount*: clean values but highly skewed (min: 0.01 and max: 3520), a logarithmic transformation was the solution to reduce the effect of the extreme values;
- *spending_deviation_score* and *geo_anomaly_score*: already normalized between fixed ranges;
- *velocity_score*: integer evenly spaced values, will be treated as numeric or one-hot encoded, depending on the model;
- *time_since_last_transaction_imp*: although missing and negative values were handled, the column remains highly skewed. So a logarithmic transformation will be applied as well.

Furthermore, to prepare categorical data for the machine learning models, we evaluated the **cardinality** and **predictive relevance** of each column. Several high cardinality features were pointed out, such as *sender_account*, *receiver_account*, *device_hash* and *ip_address*, each with hundreds of thousands to millions of unique values [9]. Applying One Hot encoding here would not be advised, so we opted for **Feature Hashing**, which maps these features into fixed length vectors, efficiently preserving the values.

In contrast, low cardinality features like *transaction_type*, *merchant_category*, *location*, *device_used* and *payment_channel* were handled using a standard **StringIndexer** to **OneHotEncoder** pipeline, configured with `handleInvalid = 'keep'` to maintain the robustness during validation and testing.

Finally, *transactions_id* was excluded from the feature set as it functions purely as a primary key with no predictive information.

2.3 Feature Engineering

2.3.1 Time Derived columns

To capture temporal patterns in the user's behavior and fraud risk, we enriched our dataset with a set of time derived features, extracted from the *event_time* column, such as:

- *hour_of_day* (0–23): captures potential **late-night transaction spikes** or anomalies related to sleep-cycle disruptions;
- *day_of_week* (1–7): helps detect patterns like **Friday night or Sunday anomalies**, which are commonly flagged in fraud analytics;
- *is_weekend* (binary): a compact signal flagging high-variance days;
- *month* (1-12): used to identify **seasonal behavior**, such as increased activity during promotional events or holidays.

2.3.2 Rolling Behavioural Metrics

In addition to the Time Derived columns, we engineered **rolling behavioral features** to capture short-term user activity dynamics, so with PySpark's window functions, we calculated metrics such as:

- *txns_1h*: number of transactions initiated by a sender in the past hour;
- *amt_24h*: total amount spent in the last 24 hours;
- *anuq_rcv_7d*: number of unique receivers in the last 7 days;
- *avg_gap_mins_10*: average time gap, in minutes, between the last 10 transactions for a given sender;

These serve as velocity and burstiness indicators, which are often more effective in the action of exposing fraudulent behaviour than static regular attributes.

2.4 Pipeline

To ensure consistency across all phases of the project, we integrated this preprocessing into a unified Spark ML Pipeline. Including the logarithmic transformations for skewed features, categorical encoding via OneHotEncoder when handling low cardinality features and FeatureHasher for high cardinality ones.

All of the features were then assembled into a single *features_raw* vector using the VectorAssembler, scaled with Standard Scaler as support for the distance based models. The target *is_fraud* was also cast to double type and class imbalance was addressed as a 26:1 weight ratio to fraudulent vs legitimate transactions when training..

2.5 Feature Selection

To reduce dimensionality and focus on the most informative inputs, we applied a Random Forest Classifier feature selection, training a 100-tree model on the preprocessed training set and extracting the top 10 most important features regarding the model's internal feature importance scores. These were saved using a VectorSlicer into a new feature column, *features_topK*, as part of a reusable pipeline for downstream model training and evaluation.

3. METHODOLOGY

The group used the preprocessing pipeline they had created to attack the classification task using several machine learning models, such as Random Forests, Gradient-Boosted Trees, Logistic Regression, and Categorical Boost Classifier. Making it possible to assess various methods to detect fraudulent transactions.

Gradient-Boosted Trees (GBT): uses an ensemble approach to build a series of weak learners (typically shallow trees), where each tree corrects the errors of the one before by minimizing a loss function. On imbalanced datasets, GBTs are particularly useful for binary classification problems due to their capacity to capture intricate, nonlinear relationships and interactions between features. GBT can be implemented using distributed training.

Categorical Boosting (CatBoost): is an ensemble of decision trees sequentially, in which the residual errors of the one before are corrected by the tree that comes afterwards. CatBoost supports distributed training, making it really good for the project, handling categorical features without the need for exhaustive preprocessing and is particularly effective for classification tasks with high-cardinality categorical data such as this one.

Logistic Regression: a light, understandable, linear model used for binary classification due to its lightness, understandability, and probabilistic output. It is the log-odds of the target class represented as a linear function of the input features. Although linear, it performs adequately when features are linearly separable and performs adequately if paired with good feature engineering. Logistic Regression is also distributed data-optimized.

Random Forest: another ensemble learning algorithm that builds numerous Decision Trees and combines their predictions via majority vote for classification. Random Forests are also robust to noise and overfitting and can handle missing values effectively, as well as provide feature importance rankings. They generalize well to diverse datasets and are built to scale well with distributed data.

4. RESULTS AND EVALUATION

4.1 Offline Results

Results were similar throughout the four tested models, with an inability to correctly identify the fraudulent transactions. The models that better predicted this class (1) were the logistic regression, achieving a F1-score of 0.073, and the random forest, with 0.0815, already very low scores and furthermore it became a trade-off with class 0 as it only achieved 0.6768 and 0.7236, respectively, lower than the other models.

The model with the highest weighted F1-Score was GBT, achieving an impressive 0.9462, however, it is unfortunately explained by over-representing class 0, with 0.9998 recall and with an F1-score of only 0.0004 for class 1, being able to correctly predict only 5 cases of this class.

Given these facts, the preferred models are either the logistic regression or the random forest, as, even though they have a lower weighted F1-score, they are better at predicting the fraudulent transactions.

We can safely conclude that these results are underwhelming and can be explained by the complexity of the dataset, in accordance with previous works around this data that faced similar issues.

4.2 Distributed Streaming Implementation

The team focused on building an online streaming of data so that real time data can be transmitted and predicted to be fraudulent or not.

As there is no actual real time data to support the project, the team used part of the available data to act as real time data arriving. This data is sent to Kafka[10][11], a distributed event streaming platform that is designed to transmit real-time streaming data that for this project

we run locally on docker and expose it to the internet. To simulate the incoming data we used a python script that platforms the data received in a .csv file into JSON kafka messages and is programmed to be sent in fixed interval periods (2 rows per second was the chosen default).

After running the script, a databricks cluster reads the streaming data from the kafka topic, converting it back into readable dataframes. This data passes through the pre-processing and feature selection pipelines and is then ready to be predicted.

After prediction, data is sent to another Kafka topic that we used to build a live dashboard which updates every second with the new information received, the dashboard is built using both dash and plotly for smooth visualizations which include a bar chart for the transactions by location complemented by the a table with the recent history of transactions, a scatter plot to visualize the relationship between transaction amount and the Geo anomaly score and a line chart that tracks how many transactions were flagged as fraudulent as a percentage over time. Besides tracking the transactions, we also provide a tab for real time analysis of model performance where we are able to track all current values for macro f1-score, precision and recall.[12][13]

After running the system until all samples were streamed, we were able to find the model that achieved the highest values of our chosen metrics was our optimized random forest, getting a precision of 0.9342 means that when our model flags a transaction as fraudulent, it is correct over 93 % of the time. This high value is not reflective of the model's quality thanks to the imbalance present in our data. A recall of 0.6875, however, tells us that the model only catches about 69 % of all actual fraud cases. In other words, roughly 31 % of fraudulent transactions still slip through undetected.

At last, the F1-score for this test was 0.7854, which is balanced but with room for Improvement. An F1 around 0.79 indicates a reasonable overall balance, but highlights that boosting recall would substantially lift the F1.

5. CONCLUSION

Through the project, a pipeline capable of handling big data for fraud detection purposes was created. Although the team extensively tried to improve the preprocessing and feature selection, the models did not produce the desired results, especially in the class for fraudulent transactions, either way the work done highlights the potential there is for ensemble methods in fraud detection analytics. The streaming procedure is working well and can be expanded to incorporate real data in a future application of this project. The dashboard is informative and adds value to information extraction.

Future work to explore could be made by testing different models such as Isolation Forests to explore anomaly detections, and further data enrichment through external datasets to improve accuracy. There is the possibility to implement security and governance mechanisms for a product-grade system with login and authentication methods as well as compliance with data privacy regulations such as GDPR. Finally, enhancements on the interface can be done through, for example, alerting systems for high confidence fraud cases.

Appendix

```
data.select(avg(col("is_fraud").cast("double")) * 100).alias("fraud_percentage").show()
```

(avg(CAST(is_fraud AS DOUBLE)) * 100)
3.59106

[1] Percentage of frauds in the whole dataset

```
# Check for duplicate transaction IDs (should be unique per row)

dups = (data_nofraud
        .groupBy("transaction_id")
        .agg(count("*").alias("cnt"))
        .filter(col("cnt") > 1))

dups_count = dups.count()
print(f"Duplicate transaction_id's: {dups_count:,}")
```

▶ dups: pyspark.sql.dataframe.DataFrame = [transaction_id: string, cnt: long]

Duplicate transaction_id's: 0

[2] Number of duplicates in the dataset

```
print("Row count:", data_nodup.count())
data_nodup.printSchema()
```

Row count: 5000000

```
root
|-- transaction_id: string (nullable = true)
|-- timestamp: timestamp (nullable = true)
|-- sender_account: string (nullable = true)
|-- receiver_account: string (nullable = true)
|-- amount: double (nullable = true)
|-- transaction_type: string (nullable = true)
|-- merchant_category: string (nullable = true)
|-- location: string (nullable = true)
|-- device_used: string (nullable = true)
|-- is_fraud: boolean (nullable = true)
|-- time_since_last_transaction: double (nullable = true)
|-- spending_deviation_score: double (nullable = true)
|-- velocity_score: integer (nullable = true)
|-- geo_anomaly_score: double (nullable = true)
|-- payment_channel: string (nullable = true)
|-- ip_address: string (nullable = true)
|-- device_hash: string (nullable = true)
```

[3] Number of rows and column types in the dataset

```

▶ Jun 02, 2025 (18s) 24

# Type correction + basic sanity filters
data_typed = (
    data_nodup
    .withColumn("event_time", to_timestamp("timestamp")) # convert to timestamp
    .withColumn("velocity_score", col("velocity_score").cast("double")) # ensure consistent numeric type
)

# Drop rows with nonsensical values (sanity filters)
data_sane = (
    data_typed
    .filter(col("amount") > 0) # negative/zero amount is invalid
    .filter(col("event_time").isNotNull()) # remove rows with bad timestamps
)

print("After type fix + sanity filters:", data_sane.count())
data_sane.select("event_time", "amount", "velocity_score", "is_fraud").show(5, truncate=False)

▶ data_sane: pyspark.sql.dataframe.DataFrame = [transaction_id: string, timestamp: timestamp ... 16 more fields]
▶ data_typed: pyspark.sql.dataframe.DataFrame = [transaction_id: string, timestamp: timestamp ... 16 more fields]

After type fix + sanity filters: 5000000
+-----+-----+-----+-----+
|event_time|amount|velocity_score|is_fraud|
+-----+-----+-----+-----+
|2023-08-22 09:22:43.516168|343.78|3.0|false|
|2023-08-04 01:58:02.606711|419.65|7.0|false|
|2023-05-12 11:39:33.742963|2773.86|20.0|false|
|2023-10-10 06:04:43.195112|1666.22|6.0|false|
|2023-09-24 08:09:02.700162|24.43|13.0|false|
+-----+-----+-----+-----+
only showing top 5 rows

```

[4] Application of the sanity filters and conclusions

```

▶ Jun 02, 2025 (28s) 27

df = data_sane

# Identify numeric columns
num_cols = ["amount", "time_since_last_transaction", "spending_deviation_score",
            "velocity_score", "geo_anomaly_score"]

# Build null + NaN check for each column
null_exprs = []
for c in df.columns:
    if c in num_cols:
        # For numeric: check both isNull and isnan
        null_exprs.append(_sum((col(c).isNull() | isnan(c)).cast("int")).alias(c))
    else:
        # For all others: only isNull
        null_exprs.append(_sum(col(c).isNull().cast("int")).alias(c))

# Run audit
null_counts = df.select(null_exprs)
print("Null / NaN count per column:")
null_counts.show(vertical=True)

▶ df: pyspark.sql.dataframe.DataFrame = [transaction_id: string, timestamp: timestamp ... 16 more fields]
▶ null_counts: pyspark.sql.dataframe.DataFrame = [transaction_id: long, timestamp: long ... 16 more fields]

Null / NaN count per column:
-RECORD 0-----
transaction_id      | 0
timestamp           | 0
sender_account      | 0
receiver_account    | 0
amount              | 0
transaction_type     | 0
merchant_category   | 0
location            | 0
device_used         | 0
is_fraud            | 0
time_since_last_transaction | 896513
spending_deviation_score | 0
velocity_score      | 0
geo_anomaly_score   | 0
payment_channel     | 0
ip_address          | 0
device_hash         | 0
event_time          | 0

```

[5] Missing values in the dataset

```

▶ Jun 02, 2025 (30s) 31 Python
total = data_sane.count()
neg = data_sane.filter(F.col("time_since_last_transaction") < 0).count()
print(f"Negatives: {neg:}, { (neg/total:.2%)}")

Negatives: 2,051,331 (41.03%)

```

[6] Number of invalid (negative) values for time_since_last_transaction

```

▶ Jun 02, 2025 (16s) 36
dq = (data_imputed
      .agg(
        F.count('*').alias('rows'),
        F.approx_count_distinct('sender_account').alias('unique_senders'),
        F.approx_count_distinct('receiver_account').alias('unique_receivers'),
        F.avg(F.col('is_fraud').cast('double')).alias('fraud_rate') # average == prevalence
      )
      .withColumn('fraud_percentage', F.col('fraud_rate') * 100)
      .drop('fraud_rate'))

dq.show(truncate=False)

dq: pyspark.sql.dataframe.DataFrame = [rows: long, unique_senders: long ... 2 more fields]
+-----+-----+-----+-----+
|rows|unique_senders|unique_receivers|fraud_percentage|
+-----+-----+-----+-----+
|5000000|874981|870056|3.59106|
+-----+-----+-----+-----+

```

[7] Descriptive statistics after duplicates and missing/strange values

```

▶ Jun 02, 2025 (27s) 39
data_imputed.select(num_cols).summary("count", "min", "25%", "50%", "75%", "max", "stddev").show()

+-----+-----+-----+-----+-----+-----+
|summary|amount|spending_deviation_score|velocity_score|geo_anomaly_score|time_since_last_transaction_imp|
+-----+-----+-----+-----+-----+-----+
|count|5000000|5000000|5000000|5000000|5000000|
|min|0.01|-5.26|1.0|0.0|0.00109436805555...|
|25%|26.56|-0.68|6.0|0.25|1454.099700473889|
|50%|138.66|0.0|11.0|0.5|2566.543702108889|
|75%|503.79|0.67|16.0|0.75|3925.8361853491665|
|max|3520.57|5.02|20.0|1.0|8777.814181944444|
|stddev|469.9333110659362|1.0008069793965741|5.766842438453673|0.2886349350409801|1875.9850900887013|
+-----+-----+-----+-----+-----+-----+

```

[8] Distribution of the key numerical features

```

▶ Jun 02, 2025 (3m) 42
[(c, data_imputed.select(c).distinct().count()) for c in cat_cols]

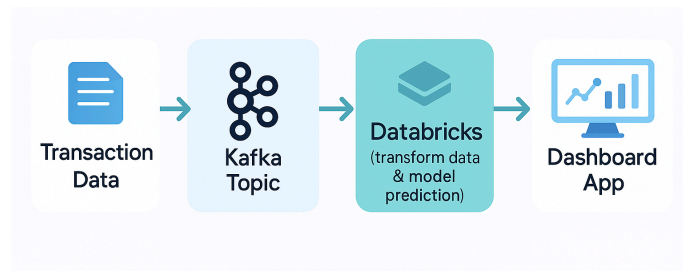
Out[17]: [('transaction_id', 5000000),
('sender_account', 896513),
('receiver_account', 896639),
('transaction_type', 4),
('merchant_category', 8),
('location', 8),
('device_used', 4),
('payment_channel', 4),
('ip_address', 4997068),
('device_hash', 3835723)]

```

[9] Distinct value counts for categorical columns

[10]-<https://spark.apache.org/docs/latest/streaming/structured-streaming-kafka-integration.html>

[11] - <https://huzzefakhan.medium.com/apache-kafka-in-python-d7489b139384>



[12] Flow of data streaming

[13]- Live dashboard demo-

https://www.dropbox.com/scl/fi/iesii0wglm6095wmapigs/full_demo.mp4?rlkey=8bn74ex32tj8xilmx8tofzttr&st=h15dgcrz&dl=0